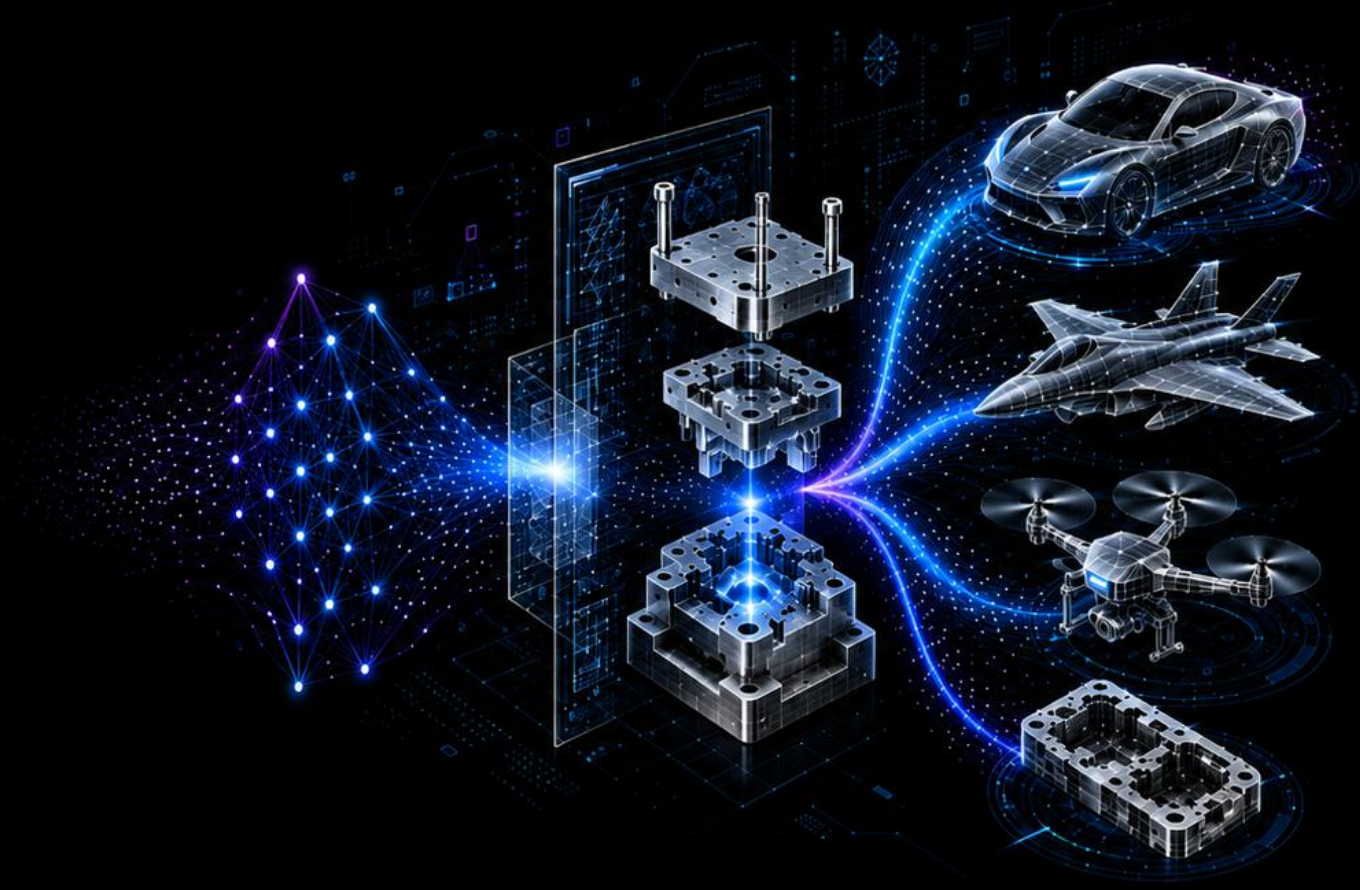
정규 스터디

8장
성능 최적화



최민혁

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Optimizer

- Optimizer: Weight와 bias를 수정하는 수학적 기법
- Gradient descent에서 gradient 항, gradient 평가 시점, learning rate를 가공

Optimizer

❖ Loss Landscape / 수렴 속도 vs 일반화 성능 / 계산 자원 / Model Architecture 별 관행에 따라 선정

❖ 학습 중간에 optimizer를 바꾸는 기법(epoch / 조건문)

```
import torch.optim as optim

# 모델 정의 후...
optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999), eps=1e-8)
```

```
switched = False
for epoch in range(total_epochs):
    train_one_epoch(model, optimizer)
    val_loss = evaluate(model)

    if not switched and val_loss_plateaued(val_loss):  # 예: 3 epoch 연속 개선 없음
        optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
        switched = True
```

❖ Scheduler: Learning rate를 조절하는 함수

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Optimizer

Optimizer

Optimizer

경로	옵티마이저	업데이트 식	특징
	SGD	$\theta_{t+1} = \theta_t - \eta \cdot g_t$	매 스텝 순간 기울기만 사용. 협곡형 지형에서 지그재그로 진동하며 느리게 수렴
	Momentum	$v_t = \beta v_{t-1} + (1-\beta) g_t$ $\theta_{t+1} = \theta_t - \eta \cdot v_t$	과거 기울기의 이동평균(관성)을 누적. 진동은 상쇄되고 곡선을 그리며 가속 수렴
	AdaGrad	$G_t = G_{t-1} + g_t^2$ $\theta_{t+1} = \theta_t - \eta / \sqrt{(G_t + \varepsilon)} \cdot g_t$	기울기 제곱을 계속 누적해 학습률을 나눔. 스텝이 갈수록 작아져 목표 전에 정체되기 쉬움
	RMSProp	$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2$ $\theta_{t+1} = \theta_t - \eta / \sqrt{(E[g^2]_t + \varepsilon)} \cdot g_t$	기울기 제곱을 지수이동평균으로 관리해 오래된 정보는 잊음. 학습률이 0으로 죽지 않아 끝까지 도달
	Adam	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ $\theta_{t+1} = \theta_t - \eta / \sqrt{(\hat{v}_t + \varepsilon)} \cdot \hat{m}_t$	Momentum(방향)+RMSProp(스텝 크기) 결합. 부드러운 궤적으로 빠르고 안정적으로 수렴

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Normalization

- Normalization: 데이터의 분포를 조절
- 학습 시 parameter를 동시에 수정하기에, 수정 후 노드의 입력 분포는 수정 전과 다름. Parameter 수정 결과가 의도와 달라짐.

Normalization

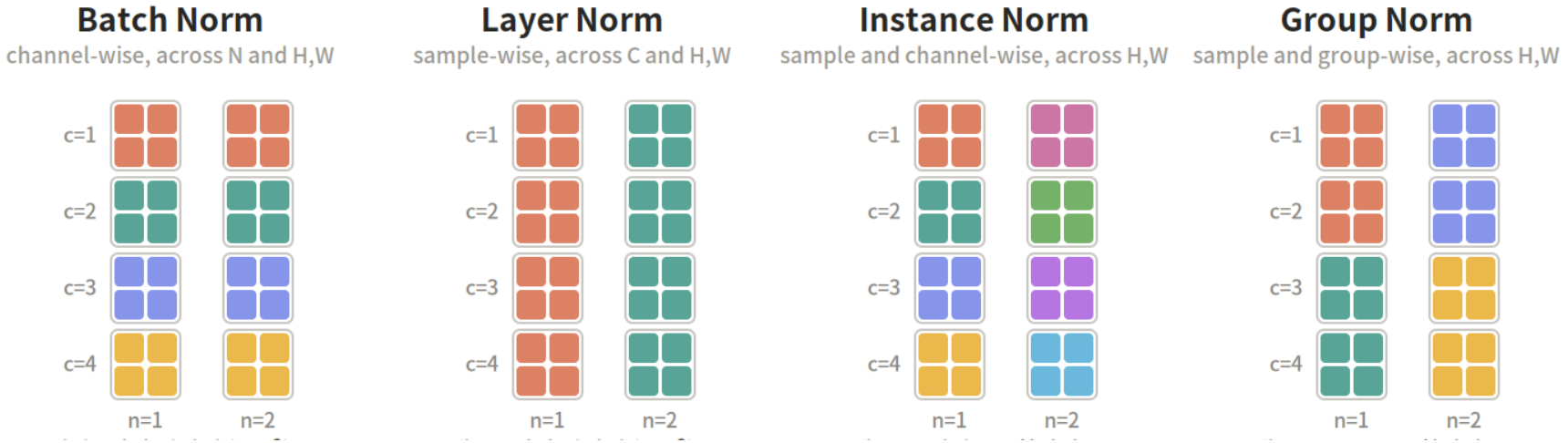
❖ 다른 layer는 함께 normalization을 진행하지 않음

❖ Batch: 각 batch에 속하는 데이터를 묶어 노드 각각을 normalization

❖ Layer: 각 데이터에 속하는 node끼리 normalization

❖ Instance: 하나의 데이터에 대해 한 채널에서 normalization

❖ Group: 한 데이터에서 각 레이어에 있는 채널을 그룹으로 나누어 그룹별로 normalization



Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Normalization

Normalization

❖ 공통수식:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\mu_{\mathcal{S}} = \frac{1}{|\mathcal{S}|}\sum_{i \in \mathcal{S}} x_i$$

$$\sigma^2_{\mathcal{S}} = \frac{1}{|\mathcal{S}|}\sum_{i \in \mathcal{S}} (x_i - \mu_{\mathcal{S}})^2$$

$$y_i = \gamma \hat{x}_i + \beta$$

❖ γ, β: 학습이 진행되는 learnable parameter

```python
import torch
import torch.nn as nn


class MLP(nn.Module):
    def __init__(self, in_dim=10, hidden_dim=32, out_dim=1):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)   # PyTorch 내장 배치 정규화
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, out_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Regularization & Generalization

- Regularization: Train 과정에 제약을 가하는 기술적 기법
- Generalization: Train data, test data를 포함해 모든 data에 대한 성능이 좋게 하는 것 (과적합 예방)

Regularization & Generalization

❖ Dropout: epoch마다 뉴런을 확률적으로 비활성화해 특정 node에 대한 의존성을 낮추는 기법

❖ Weight Decay: 가중치가 커지는 것 자체에 벌점을 줘서, 모델이 지나치게 복잡한 함수를 학습하지 못하게 막는 기법

$$L(w) = L_0(w) + \frac{\lambda}{2}\sum \mathrm{w}_\mathrm{i}^2$$

```
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
```

❖ Early Stopping: Epoch를 진행하며 validation Data에 대한 loss가 줄어들지 않을 때 학습을 조기에 종료하는 기법

❖ Data Augmentation: Train data에 변형을 가해 수와 다양성을 인위적으로 늘리는 기법

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Regularization & Generalization

Regularization & Generalization

Early Stopping

```
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0):
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    def __call__(self, val_loss):
        # 오차가 감소했을 경우 (성능 개선)
        if val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0  # 카운터 초기화
        # 오차가 감소하지 않았을 경우
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
# 조기 종료 객체 생성 (5번 연속 개선 없으면 종료)
early_stopping = EarlyStopping(patience=5)

for epoch in range(num_epochs):
    # train(...)
    # val_loss = validate(...)
    early_stopping(val_loss)
    if early_stopping.early_stop:
        print(f"{epoch}번째 에포크에서 조기 종료 조건 만족. 학습을 중단합니다.")
        break
```

Data Augmentation

```
import torchvision.transforms as transforms
from torchvision import datasets

# 1. 데이터 증강 파이프라인 정의 (훈련 데이터용)
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),      # 무작위 영역을 자른 후 224x224 크기로 조절
    transforms.RandomHorizontalFlip(p=0.5), # 50% 확률로 이미지 좌우 반전
    transforms.RandomRotation(degrees=15),  # -15도 ~ 15도 사이에서 무작위 회전
    transforms.ColorJitter(brightness=0.2), # 밝기를 무작위로 미세 조절
    transforms.ToTensor(),                  # 이미지를 PyTorch 텐서로 변환 (0~1 사이 값)
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]) # ImageNet 기준 정규화
])

# 2. 데이터셋을 불러올 때 transform 인자로 전달하여 적용
# train_dataset = datasets.ImageFolder(root='./data/train', transform=train_transform)
```

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Compression

- Compression: 모델의 parameter 수나 연산량을 줄여서 저장 공간과 연산 비용을 낮추는 것

Compression

❖ **Pruning**: 가중치가 작은 parameter와 뉴런 제거
- ✓ Structured pruning: 구조 단위(node, fillter 등)를 제거해 연산이 효율적인 형태로 남김

❖ **Quantization**: Parameter와 node 출력을 저정밀 데이터로 이산화
- ✓ 기법: PTQ(학습이 끝난 후 적용), QAT(학습과정에서 양자화로 인한 오차를 반영. 정확도 손실 최소화)

❖ **Knowledge Distillation**: Teacher model이 예측한 soft label로 student model이 학습

❖ **Low-rank factorization**:

$$W = UV, U \in R_{(m\times r)}, V \in R_{(r\times n)}$$

- ✓ 파라미터 수: $mn \rightarrow r(m+n)$
- ✓ 에너지 보존비율: $\boldsymbol{E(r)} = \frac{(\|\boldsymbol{W}\|_F^2)}{\|\boldsymbol{W_r}\|_F^2}$
- ✓ *LoRA: Fine-Tuning에 활용*

$$\Delta W \approx BA$$
$$h = W_0 x + \Delta W x = W_0 x + BAx$$

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Compression & Optimization

Quantization

❖ 실수 구간: $[\alpha, \beta]$

❖ $b\ bit$

- ✓ Scale: $s = (\beta - \alpha)/(2^b - 1)$
- ✓ Zero-point: $z = round(-\alpha / s)$ 신경망에서 0은 "이 뉴런이 완전히 꺼졌다"는 의미, *0을 구현하는 것이 중요*
- ✓ Quantization: $q = clip(round(x/s) + z, 0, 2^b - 1)$
- ✓ Dequantization: $\hat{x} = s(q - z)$

❖ 대칭 양자화: $\beta = -\alpha = \max(|x|),\ z = 0$

- ✓ z 더하는 계산이 없어짐
- ✓ ReLU의 output은 음수가 없음 -〉 편향된 분포에서 비효율적

❖ QAT

- ✓ 학습 순전파 시 dequantization한 parameter를 사용
- ✓ 역전파 시 round()를 항등함수로 근사

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Compression & Optimization

Quantization

```
# 1. 다채널 센서 데이터를 처리할 간단한 ANN 모델 정의
class SensorANN(nn.Module):
    def __init__(self):
        super(SensorANN, self).__init__()
        # 양자화 및 역양자화 스텁: 텐서의 타입을 변환하는 관문 역할
        self.quant = quant.QuantStub()

        self.fc1 = nn.Linear(8, 16)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(16, 2)

        self.dequant = quant.DeQuantStub()

    def forward(self, x):
        x = self.quant(x)      # FP32 -> INT8 변환
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.dequant(x)    # INT8 -> FP32 변환
        return x
```

일반적인 NN의 학습 진행

```
# 모델 인스턴스화 및 평가 모드 전환 (양자화는 추론 단계 최적화 기법임)
model = SensorANN()
model.eval()

# 2. 양자화 엔진 설정 (ARM 기반 AIoT 기기를 가정하여 'qnnpack' 백엔드 사용)
torch.backends.quantized.engine = 'qnnpack'
model.qconfig = quant.get_default_qconfig('qnnpack')

# 3. 양자화 준비 (옵저버 삽입)
# 텐서의 최솟값(min)과 최댓값(max)을 관찰할 수 있도록 모델 내부에 Observer를 삽입합니다.
model_prepared = quant.prepare(model)

# 4. 캘리브레이션 (Calibration)
# 대표적인 센서 데이터 샘플을 통과시켜 데이터의 실제 분포 범위를 측정합니다.
# 이 과정을 통해 앞선 공식의 Scale(s)과 Zero-point(z) 값이 수학적으로 도출됩니다.
dummy_sensor_data = torch.randn(100, 8)  # 100개의 샘플, 8개의 물리 센서 채널 데이터 가정
model_prepared(dummy_sensor_data)

# 5. INT8 양자화 변환
# 측정한 통계치를 바탕으로 32비트 부동소수점 가중치를 8비트 정수로 영구 변환합니다.
model_quantized = quant.convert(model_prepared)
```

Part 1 Optimizer & Learning Rate

Part 2 Normalization & Regularization & Generalization

Part 3 Compression

Compression & Optimization

Low-rank factorization

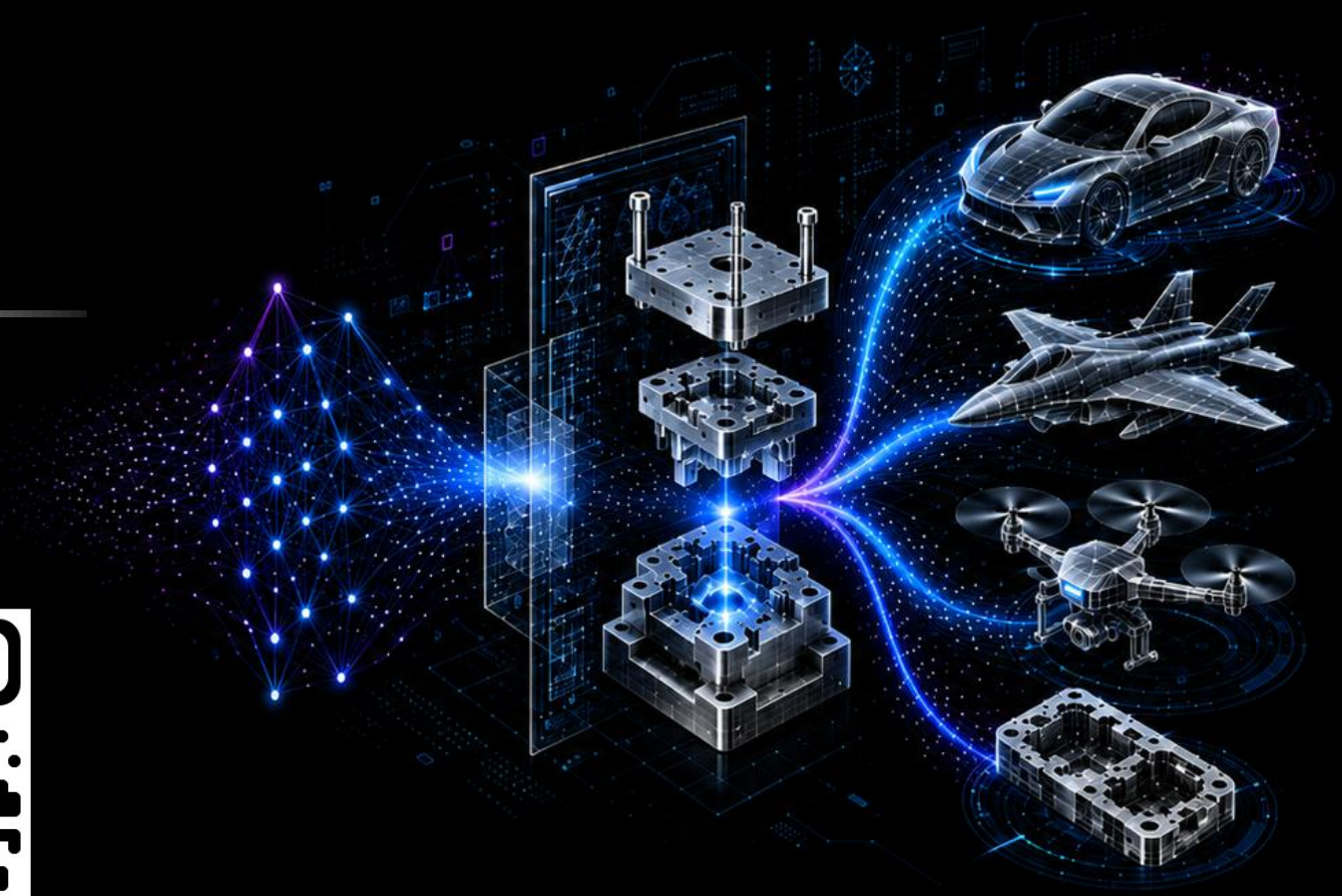
```python
class LowRankLinear(nn.Module):
    def __init__(self, in_features, out_features, rank, bias=True):
        super().__init__()
        self.V = nn.Linear(in_features, rank, bias=False)   # (256 → rank)
        self.U = nn.Linear(rank, out_features, bias=bias)   # (rank → 256)

    def forward(self, x):
        return self.U(self.V(x))

class LowRankMLP(nn.Module):
    def __init__(self, in_dim=20, hidden=256, out_dim=4, rank=16):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.fc2 = LowRankLinear(hidden, hidden, rank)
        self.fc3 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

Any Questions?



이름 최 민 혁

소속 한양대학교 기계공학과

이메일 mindat@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab